VTX Theory of Operation

How the Virtual Terrain Explorer generates and renders a procedural universe

Dean Sindorf
Draft, November 2026

Purpose and scope 2
- Motivation 2
- Design principles 2

System overview 3
- The object tree 3
- Source layout 4

Scene representation 4
- Scene files 4
- Parsing 5
- Random generation 5

Scale, coordinates and precision 6
- Units 6
- Depth groups 6
- Eye-relative rendering 6
- Double-precision noise 6

The expression system 6
- Expression trees 6
- One expression, two evaluators 6
- Noise 7
- Terrain functions 7

Adaptive terrain 7
- Mesh structure 7
- The refinement test 8
- The adapt loop 8
- Visibility 8
- Convergence 8

Placed objects 9
- Placements 9
- Craters 9
- 3D rocks 9
- Plants 9
- Sprites 9

Asteroids and marching cubes 10
- Octree 10
- Finding the surface reliably 10
- Near and far planes 11

The rendering pipeline 11
- Frame sequence 11
- Buffers 11

Shader management 11
Lighting and shadows 12
Surface lighting 12
Two kinds of shadows 12
Atmosphere and clouds 13
Sky and haze 13
Three cloud types 13
Volumetric clouds 13
Cloud shadows on the ground 14
Water and reflections 14
Stars, galaxies and nebulae 14
The user interface 15
Known limitations 15
Acknowledgments 15

Purpose and scope

This document explains how VTX works and why it is built the way it is. It is written for developers with some OpenGL and C++ experience who want to understand, modify or learn from the program. The user manual, not this document, covers how to operate VTX.

Class, file and shader names refer to the source tree. Each chapter names the files to read next.

Motivation

VTX began in 1996 as a C/C++ hobby application and has been developed in bursts ever since. Its stated motivation is:

> To be able to explore the universe (at least vicariously) in my lifetime without the need to wait for true space exploration technology to catch up or to be limited by physical constraints (such as the speed of light).

The matching objective is an application that produces a credible visual emulation of the known physical universe, from planets to galaxies, with a virtually unlimited level of detail, on a standard PC with a decent graphics card.

Design principles

Four ideas follow from those goals and shape nearly everything in the code:

- **Generate instead of store.** A universe cannot be kept on disk. Surfaces, plants and rocks are computed from short expressions, so a whole planet can be described by a few lines of text.
- **Spend detail where the viewer is looking.** Terrain, asteroids and placed objects are refined from what is on screen, and the result is discarded as the view moves on.
- **Keep scenes editable.** A scene is a readable text file. The expression that creates a surface is the same one the editor displays and saves.
- **Compute what the eye can see, not the physics.** The lighting models are physically inspired where

that buys realism, and are cheap approximations everywhere else.



Volumetric clouds over a rocky lake, rendered by VTX

System overview

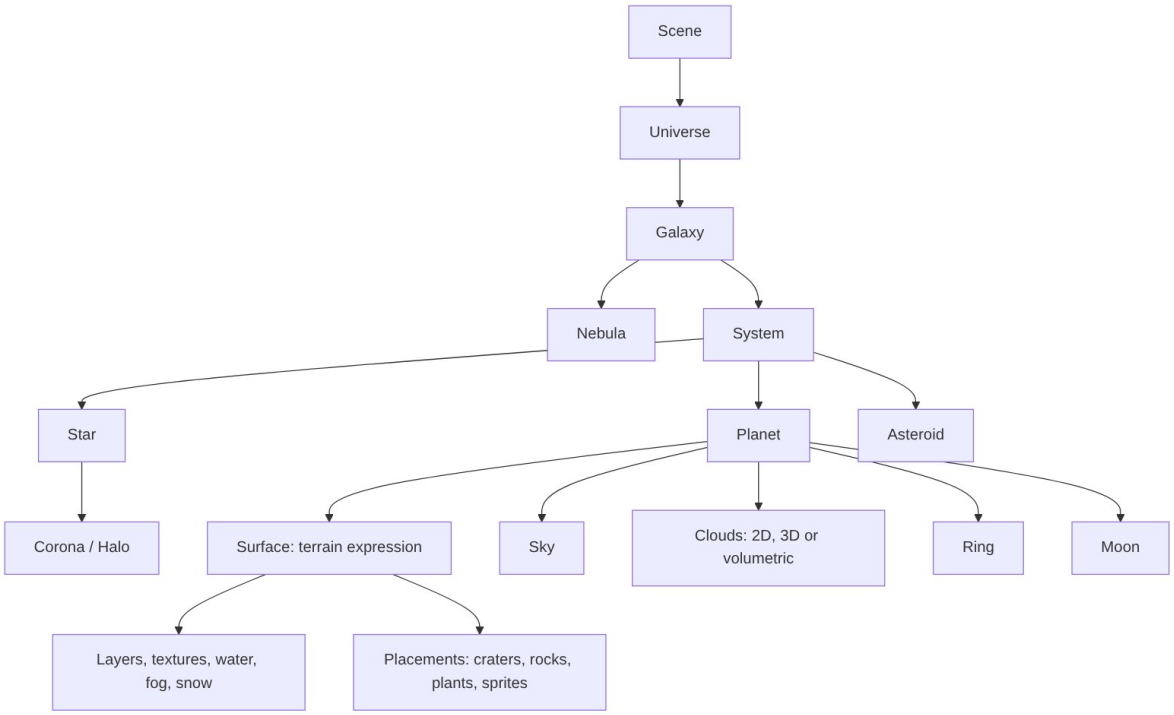
VTX is a C++ desktop application. OpenGL and GLSL do the rendering, wxWidgets provides the interface, and flex and bison implement the scene language. It is developed on Windows with Cygwin and MinGW tools, and has also run on Linux.

At the highest level, each frame does three things:

1. **Adapt:** update procedural data structures for the current view.
2. **Render:** draw every visible object into off-screen buffers, from far to near.
3. **Apply effects:** add water, fog, haze, late cloud passes and tone mapping as screen-space passes.

The object tree

Everything in a scene lives in one tree. The tree mirrors the physical nesting of the universe, and the same structure appears in scene files and in the editor’s Scene Tree dialog.



The main levels of the scene object tree

All visible objects derive from `Object3D` and `ObjectNode` (`ObjectClass.h`). `Orbital` adds orbital behavior, and `Spheroid` adds a sphere with lighting and optional terrain. `Planetoid` is the base of `Planet` and `Moon`, while `Star`, `Asteroid` and `Shell` (the base of `Sky`, `Halo`, `Corona`, `CloudLayer` and `Ring`) are other `Spheroid` branches. `Scene` (`SceneClass.cpp`) owns the tree and the frame loop.

Source layout

Location	Contents
`includes/`	The core engine: objects, terrain, noise, rendering
`wx/`	The wxWidgets interface, including one tab class per editable object type
`parsers/`	The scene language: `sx.l` (flex) and `sx.y` (bison)
`Shaders/`	GLSL vertex, geometry and fragment shaders, and shared includes
`Saves/`	Saved scenes (`.spx`)
`Textures/`, `Images/`, `Resources/`	Image assets, screenshots and UI resources

The largest and most central files are `OrbitalClass.cpp` (the object types), `SceneClass.cpp` (the frame loop), `MapNode.cpp` and `MapClass.cpp` (adaptive terrain), `TerrainNode.cpp` and `TerrainClass.cpp` (the expression tree) and `Raster.cpp` and `Effects.cpp` (shadows and screen-space passes).

Scene representation

Scene files

A scene is a text file with the extension `.spx`. It holds saved views followed by the object tree. This is

abbreviated from `Saves/alpine.spx`:

```
Scene() {
    View{
        view=SURFACE;
        height=13131.5 ft;
        tilt=-17;
        fov=35;
        viewobj=Universe.1.Galaxy.1.System.1.Planet.1;
    }
    Universe {
        Galaxy(20000) {
            noise.expr=noise(GRADIENT|NLOD,2,7,1,0.2)+0.1*noise(GRADIENT|NLOD,14,5,0.5);
            System(1000) {
                Star(0.5,0) { temperature=6000; Corona(4) { ... } }
                Planet(0.004,100) {
                    Surface{
                        terrain=Z(noise(GRADIENT|NABS,8,14.1,0,1,2.1,2,0.11,0,-0.3))+
                                Texture("alpine",BORDER|LINEAR|
TEX,2048,2,2,0,1,1,1,0,0,0.80952,0.3,-0.36508);
                    }
                    Sky(0.00419081) { ... }
                }
            }
        }
    }
}
```

The file format is the program's real data model. `Scene::save` and the per-class `save` methods write the tree back in the same language, so a scene can be edited by hand or in the interface, and switched between the two.

Parsing

`parsers/sx.l` tokenizes the text, and `parsers/sx.y` defines the grammar for scene items, views and each object type. Parser actions construct `Orbital` objects and `TNode` expression nodes directly. The editor reuses this parser: `TheScene->parse_node()` turns a string into a node, so editing a parameter generally means rebuilding the text of an expression and parsing it again.

Random generation

Going to an unexplored star, or using a Generate command in the Scene Tree dialog, creates objects without any scene file. Generators such as `Planetoid::newRocky`, `newGasGiant` and `randFeature` assemble the same expression text a person could write, using prototypes and randomized parameters. Because the output is an ordinary expression, a generated world can be inspected, edited and saved like any other.

Scale, coordinates and precision

Modelling a galaxy and a pebble in one program is mostly a numerical problem, and several VTX mechanisms exist only to solve it.

Units

One scene unit is one million miles (`defs.h`). `FEET` and `MILES` are constants for converting into scene units, so a foot is about 1.9e-10 units. Scene files accept suffixes such as `ft`.

Depth groups

No single depth range can hold a nearby rock and a distant moon. Each object is therefore assigned to a group (`BG6` through `BG0`, then `FG1` and `FG0`) based on its distance. `Scene::render_objects` draws the groups from furthest to nearest and clears the depth buffer between them, with each group using its own near and far planes. Stars and galaxies draw first as a backdrop.

Eye-relative rendering

GPU geometry uses single-precision floats, which cannot hold planetary coordinates to sub-foot accuracy. Vertices are sent relative to the eye, so values stay small and accurate where the viewer can see them.

Double-precision noise

Fragment-shader noise has the same problem: at planetary scale, a float position loses enough precision that noise turns blocky. When `NOISE_DP` is on, `MapNode` and `noise_funcs.h` carry a double-precision noise point through the shaders, using GLSL `double` where the GPU supports it. It is more expensive, so it is compiled in only where needed: planets, 2D and volumetric clouds, and the geometry-shader terrain path.

The expression system

Almost every procedural value in VTX is an expression tree, which is the program's central abstraction.

Expression trees

`TNode` (`TerrainNode.h`) is the base class for all nodes. There are nodes for constants, arithmetic and comparisons, conditionals (`TNcond`), noise (`TNnoise`), geometry (`TNpoint`, written `Z(...)` in a scene file), colors (`TNcolor`), textures (`TNtexture`), and features such as `TNwater`, `TNfog`, `TNsnow`, `TNcraters`, `TNrocks` and `TNplant`. A surface is one expression that combines these nodes with `+`.

`TerrainData` (`TerrainData.h`) is the accumulator: as a tree is evaluated, it collects height, color, density, ocean state, erosion and more. `TerrainMgr` and `ExprMgr` hold expression scopes and variables.

One expression, two evaluators

A key design feature is that a single expression can run in either of two places:

- **On the CPU, per vertex:** the height (`Z`) of terrain has to exist before geometry can be built, so `Map` evaluates the tree at each node it creates. Results are stored in `MapData`.
- **On the GPU, per pixel:** an expression that can run per pixel is translated to GLSL source. For example, `TNpoint::initProgram` emits `#define NPZ <expression>`, and `TNcolor` emits `#define NCC vec4(...)`, using each node's `valueString`. Shaders then simply use `NPZ` and `NCC`.

Translating expressions to source means a shader program depends on the scene content, so `GLSLMgr` caches compiled programs by their defines and shader names.

Noise

`Noise` (`NoiseClass.h`) implements gradient, Voronoi and Simplex noise in 1 to 4 dimensions, with options controlling octaves, absolute value, squaring and more. `Noise` holds a large permutation table, and CPU and GPU versions (`noise_funcs.h`, `noise.h`) use matching forms so a pattern can be evaluated on either. `NLOD`-style options limit octaves to what a given distance can resolve. Shader helpers such as `belts`, `zonal`, `stormwarp`, `vortwarp` and `spots` provide gas giant features.

The octave limit is disabled for marching-cubes objects: corners shared by cells at different depths must get identical values, or the mesh tears.

Terrain functions

- `TNfractal` applies a fractal adjustment using neighboring heights as a node is created.
- `TNerode` (`Erode.cpp`) is not an erosion simulation. It is a single-pass, local classifier: a base height below `level` is fully incised, heights above `level+edge` are untouched, and the band between forms a cliff. It produces mesa and canyon shapes, and because it needs no neighbors, it works with on-demand generation.
- `TNmap` and `TNlayer` (`Layers.cpp`) stack multiple surface materials.
- `TNwater`, `TNsnow` and `TNfog` add a water surface, snow and height-based fog density.

Adaptive terrain

Planet surfaces are the heart of VTX. `Map` (`MapClass.cpp`), `MapNode` (`MapNode.cpp`) and `MapLink` (`MapLink.cpp`) build a view-dependent mesh over a sphere, in which cells near the viewer can be feet across while the far side of the planet stays coarse.

Mesh structure

`Map::make` starts with four nodes at the poles (two at each pole, at longitudes 0 and 180). Nodes have left, right, up and down neighbor pointers, and `MapLink` connects a node to the nodes it created. Refinement alternates between "squares" and "diamonds" passes, which is why each cycle runs `ADAPT_CYCLE` twice. Cell size at each level comes from `CELLSIZE(level)`, up to `PLVLS` (64) levels.

Each node holds a `MapData` record with its position and the values from the expression tree. Sub-surfaces such as water above land chain together through `next_surface`.

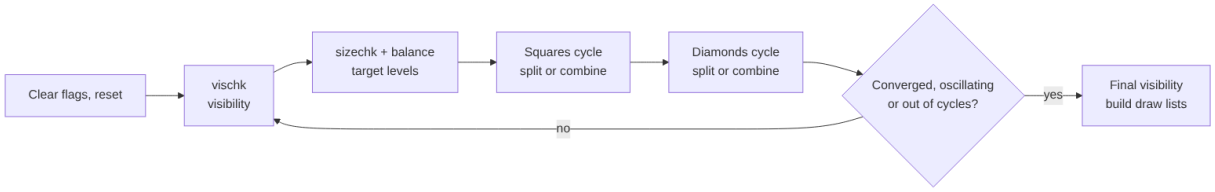
The refinement test

`MapNode::set_tests` decides what happens to a node using its size on screen:

```
ext = wscale * size / depth      // projected size in pixels
max = resolution * tlevel_factor
min = max / 5
split if ext > max
combine if ext < min
```

The factor of five between the thresholds provides hysteresis, so nodes don’t flip back and forth with small view changes. `Adapt.tlevel` lets each node’s target vary, so hidden nodes get less detail than visible ones, and `sizechk` and `balance` keep neighboring cells within a safe level difference.

The adapt loop



One Map::adapt run

`Map::adapt` is an iterative process, capped by `Adapt.maxcycles`. If it runs out of cycles, `need_adapt` stays set and work continues in the next frame, which is why a scene can sharpen progressively.

Visibility

Clipping and backface tests handle the simple cases. For the view object, an ID-buffer test handles occlusion: `Map::render_ids` draws each node in a unique color (`map_ids.frag`), and the image is read back to find which nodes actually reach the screen. Hidden terrain then gets coarser, which saves a great deal of geometry behind nearby hills.

Convergence

Visibility feeds refinement, and refinement changes visibility, so the system can fall into a loop instead of settling. `Map::adapt` guards against this in two ways:

- `adapt_osc` keeps a history of each cycle’s size and its created and deleted counts. If two consecutive cycles match those from `P` cycles earlier (for `P` up to 64), the map is oscillating and adapt stops.
- `ADAPT_VSTABLE` handles maps with few visible nodes. If the visible count holds steady for 8 cycles while the map size stays within a limited range, the remaining changes are in invisible nodes and adapt stops.

Oscillation in some orbital views is still an open problem.

Placed objects

Craters, rocks, plants and sprites all answer the same question: which objects exist near this point, on an infinite surface, without storing any of them?

Placements

`PlacementMgr` (`Placements.cpp`) answers it with a multi-scale hash grid. For a surface point it visits `levels` scales, with `size` shrinking by `mult` at each one. At each scale, the point is divided by `size` and floored to get a cell, the cell coordinates are hashed, and a `Placement` is found or created in a table with `PERMSIZE` buckets. Each placement's position and size derive from its cell, so they come out the same whenever that area is regenerated. Neighbouring cells can be included, and `Placement::set_terrain` applies an object's effect to a nearby point.

Terrain evaluation captures position, height and density at each placement (`PlaceData`), so renderers can position objects and fog them correctly without re-evaluating the terrain.

Craters

`CraterMgr` (`Craters.cpp`) uses placements to change terrain height, with jitter, radius and noise offsets drawn from each placement's hash. Its ID must be identical in every pass that evaluates the terrain, or nodes adapted later would get different craters than their neighbors.

3D rocks

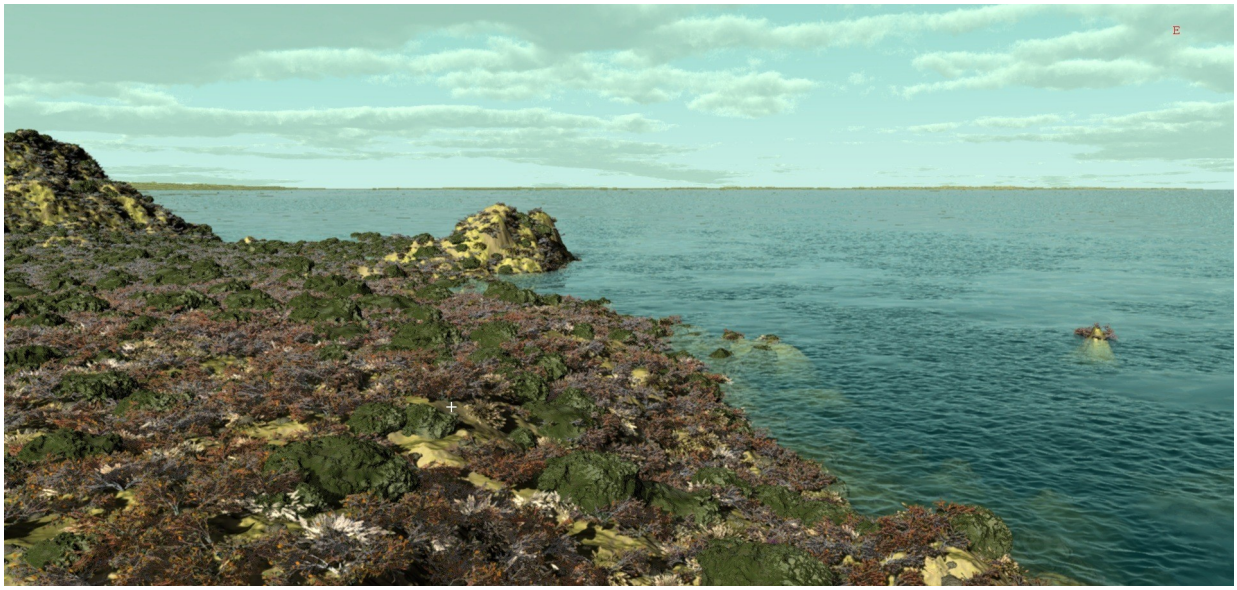
`Rock3DObjMgr` (`Rocks.cpp`) draws rocks as instanced meshes. Marching cubes builds template meshes at several resolutions, and `getTemplateForLOD` and `getLODResolution` choose among them by projected size. Instances are grouped into batches by template and layer, and drawn with `glVertexAttribDivisor`, with per-instance data including position, size and fog values. State is keyed by terrain layer ID, because `TerrainProperties` objects are rebuilt every frame.

Plants

`TNplant`, `TNBranch` and `TNLeaf` (`Plants.cpp`) describe a plant as a recursive branching process. Each level spawns children with randomized placement, curvature, width taper and leaf clusters. The geometry goes into a VBO, and geometry shaders generate the final shape: `plants3D.geom` extrudes branches into cones, while `plants2D.geom` draws flat, shaded strips that are cheaper at a distance. VBO batching took one test case from under 1 FPS to about 15 FPS.

Sprites

`SpriteObjMgr` (`Sprites.cpp`) draws point sprites from image atlases for ground cover and distant vegetation. Sprites write a type of 0 into the parameters buffer to bypass the effects pass's haze correction, because their edges would otherwise show a dark border.



Plants and rocks on a shoreline

Asteroids and marching cubes

A heightfield can’t represent overhangs, holes or irregular bodies, so `Asteroid` (`OrbitalClass.cpp`, `MCObjects.cpp`) uses a true 3D scalar field and marching cubes over an adaptive octree. The earlier standalone GLCubes project led to this design.

Octree

`MCObjTree` holds `MCObjNode` cells. Each cell adapts by its projected screen size, split and collapse update the tree, and leaves mesh themselves with `generateMesh`, using `MCGenerator`. Mesh resolution rises with screen size as well. An ID-buffer pass (`render_ids`, `mark_visibility`) finds visible leaves, and `pixelLeaf` and `raycastLeaf` resolve picks.

Finding the surface reliably

The critical question is whether a cell contains any surface, because a wrong “no” removes a whole subtree and leaves a hole. A sign test over samples can prove a surface exists, but never that one is absent, because the surface can pass between samples.

`checkSurface` therefore adds a distance-margin guard. From the field value and gradient it bounds the distance to the surface:

```
d >= |f - iso| / |grad f|
```

If `d` exceeds the sampling radius, no crossing can hide between samples and the cell is provably empty. Otherwise the answer is “unknown”, and the cell is refined instead of collapsed. The test is scale-free and only costs three extra field evaluations on cells about to be declared empty.

Near and far planes

Asteroids can fill the view at tiny scales. `Asteroid::scale` derives near and far planes from the bounds of the built mesh, and `surfaceRByRaycast` measures the real surface below the camera for altitude and camera moves.

The rendering pipeline

Frame sequence

`Scene::render` runs when something has changed: the view, time, detail or render options. It may run a couple of extra times after a move to let the display settle, and to continue an unfinished adapt.

Step	Code	What it does
1	`Scene::adapt`	Refines maps and octrees for this view
2	`Scene::shadow_group`, `Raster.renderFgShadows`	Renders depth from the light and builds the shadow image
3	`Scene::render_skyview`	Renders an extra view above the screen, when water needs it
4	`Scene::render_objects`	Draws depth groups from far to near into the image and parameter buffers
5	`drawVolume`	The last depth group of step 4: volumetric clouds over sky pixels, and cloud shadows on the ground
6	`auximage.frag`	Water pass: body color, glints and reflection ray setup
7	`effects.frag`	Traced reflections, fog and haze
8	`postEffectsHook`	Volumetric clouds over terrain, and cloud shadows on water
9	`postproc.frag`	Edge smoothing, HDR exposure and tone curve

Buffers

Rendering goes to a framebuffer object with these attachments (`GLSLMgr.h`):

Attachment	Sampled as	Contents
0	`FBOTex1`	Scene image
1	`FBOTex2`	Parameters: type (red), window depth (green), reflection term (blue), fog density (alpha)
2	`FBOTex3`	Shadow image, later the effects image
3	`FBOTex4`	Reflection ray end point, later the effects parameters
4	`FBOTex5`	Reflection ray direction and depth

A separate sky view framebuffer provides `FBOTex6` and `FBOTex7`.

The type channel lets screen-space passes treat pixels correctly: 0 is sky or background, 1 is water, 2 is ice, 3 and above is land, and 4 is rocks and plants. Pixels need a type above 0.6 to receive haze.

Shader management

`GLSLMgr` and `GLSLProgram` load, cache and bind programs. GLSL's support for `#include` was unreliable, so `GLSLProgram` expands includes itself. Each program gets a defines string assembled by `setProgram` methods, so one source file produces many variants. Expanded source is written to `.debug` files, which is useful for reading a compile error against what the driver really saw. `GLSLVarMgr` collects uniforms.

Lighting and shadows

Surface lighting

`lighting.frag` provides terrain lighting: selectable lighting models, a hemispherical ambient term and a twilight band set by `twilite_min`, `twilite_max` and `twilite_dph`. `sunlight.h` reddens direct sun near the horizon from `Sky::sunTau`, applying Rayleigh scattering's wavelength dependence along the air path. Water and ice use their own half-vector specular, in `auximage.frag`, which stretches a low sun into a long glitter path.

Two kinds of shadows

VTX uses two shadow methods, because shadows from small and large objects have very different needs.

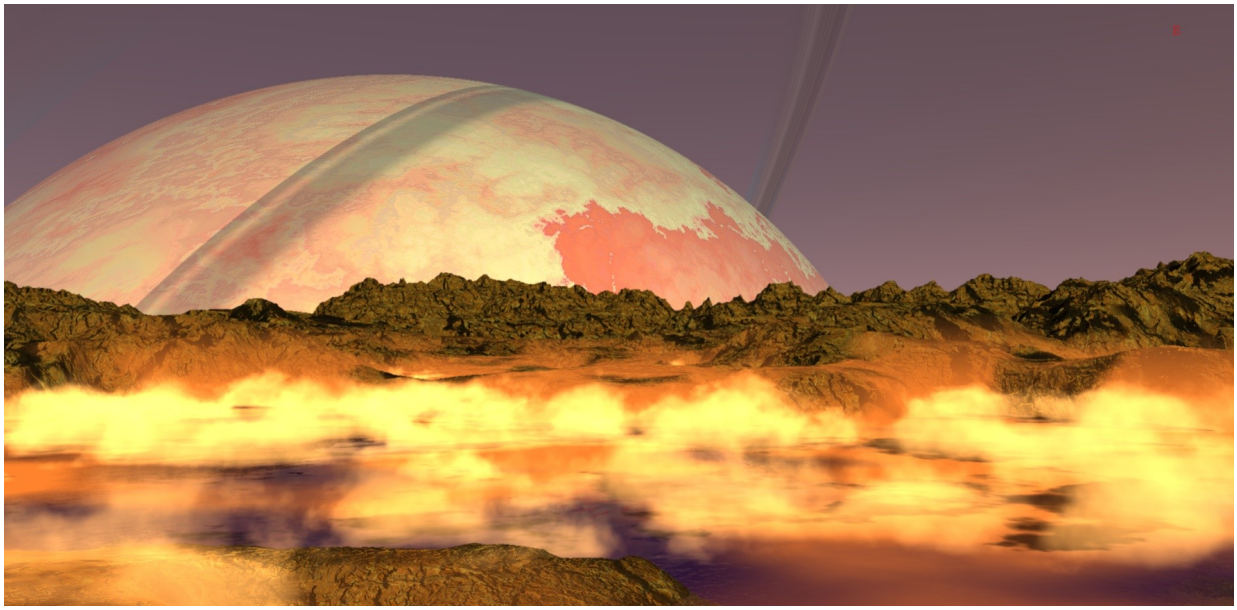
Shadow maps handle complex nearby geometry: terrain on terrain, and rocks and plants on terrain and each other. In the shader pipeline (`EffectsMgr::render_shadows`), depth is rendered from the light into a map at `shadow_map_ratio` (4) times the screen size, then compared from the eye with a jittered 64-sample filter (`shadows.frag`) to soften edges. In surface views, cascaded shadow maps with logarithmic splits spread resolution over distance (`Raster.cpp`).

Analytic shadows handle planets, moons and rings, whose shapes are known exactly and whose shadows would be coarse or absent in a map. With `analytic_shadows` on (the default), each fragment traces a ray to the sun:

- **Spheres** (`sphere_shadow.h`): the closest approach of the ray to each sphere's center gives full shadow inside the radius, with a soft edge whose width comes from the sun's apparent size at that distance. Small or distant bodies never fully cover the sun, which models the antumbra.
- **Rings** (`planet_shadows.h`): the ray's crossing of the ring plane gives a radius, and the shadow comes from the ring's opacity profile, so bands and gaps appear in the shadow. `ringProfile` builds that profile from the ring's texture. Three samples blur it by the sun's size.

Rings, and planets and moons other than the view object, then drop out of the shadow-map passes, so the map holds only the view object and what's on it. `planetShadow` is shared by terrain, rocks, plants and sprites, and `analyticShadowDefs` and `analyticShadowVars` configure the shaders.

`Shadow` color and intensity come from each body's Lighting tab. On terrain, the color acts as the light inside the shadow. Rocks, plants and sprites tint their pixels with it.



A ringed planet rising, with its ring casting a shadow across its own surface

Atmosphere and clouds

Sky and haze

`Sky` is a shell around the planet, drawn from inside or outside by `sky.front.frag` or `sky.back.frag`. Its color combines a hand-tuned system of halo, twilight, night and haze colors with `scattering.h`, an optional closed-form Rayleigh and Mie approximation scaled by `scatter_mix`. Composition drives the hue: `ghg_fraction` stands in for how much haze-forming material the air holds, which tints the Mie term, and `pressure` scales the strength.

Haze is a distance-based effect applied to terrain in `effects.frag`. Within the `haze_min` to `haze_max` band, it follows an exponential curve set by `haze_grad`, as light extinction does. That pass skips sky pixels, so clouds apply the same formula themselves through `cloud_haze.h`.

Three cloud types

Type	Technique	Main shader	Strength
2D	Expression evaluated per pixel on a shell	`clouds.2d.frag`	Fast, good from orbit
3D	Geometry shader sprites from map nodes	`clouds.3d.frag`	Puffy shape, moderate cost
Volumetric	Full-screen ray march	`clouds.volume.frag`	Most realistic, most expensive

Volumetric clouds

`clouds.volume.frag` marches each view ray through the spherical shell between the lowest cloud base and highest cloud top.

- **Density:** the layer's Point Z expression gives the shape at each 3D sample, and Color alpha can add a 2D coverage mask. Without those, a built-in shape combines Perlin and inverted Worley noise.

High-frequency Worley erosion makes tops billowy and bases wispy.

- **Light:** each sample marches toward the sun for Beer-Lambert transmittance, with several scattering octaves (`VOL_MS_OCTAVES`) standing in for multiple scattering and a two-lobe Henyey-Greenstein phase function. Sky light is occluded by the cloud above, and ground bounce lights the bases.
- **Sampling:** steps are finer near the ray's start (`stepT`), edge samples are averaged (`VOL_EDGE_SUB`) and the column coverage is reused across steps (`VOL_WSTRIDE`).
- **Distance:** fade and haze are applied per sample at that sample's own distance. A single per-ray distance makes haze jump between neighboring pixels, which appears as a sharp edge across thin clouds.

The clouds are drawn in two passes. Sky pixels are done in the main pass (`VOL_SKY`). Clouds in front of terrain must go over the hazed image, so they're drawn from the post-effects hook (`VOL_POST`). If no effects pass will run, one pass does both (`VOL_TERRAIN`).

Cloud shadows on the ground

`groundLight` marches from each terrain pixel toward the sun, sampling the same density functions and darkening the pixel by what gets through. Because this happens in the full-screen cloud pass, it covers everything already drawn, with no changes to the other shaders. Land is shadowed in the main pass, so haze goes on top of the shadow. Water isn't drawn until later, so it's shadowed in the post pass with the haze amount taken into account.

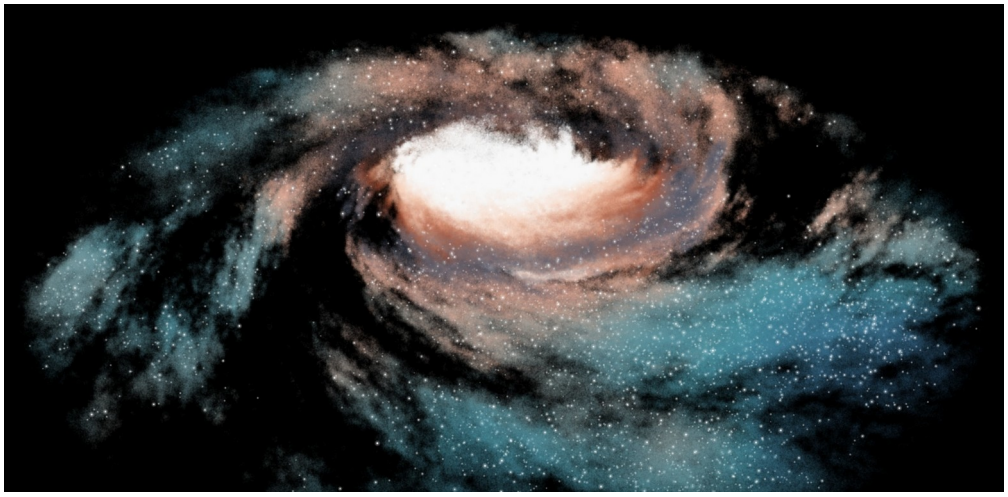
Water and reflections

Water is drawn after the scene, in its own pass (`auximage.frag`), using each node's water surface. It can read the scene image and depth behind it, so it computes clarity from the depth of water and mixes the bottom with a body color. It also adds glints and writes reflection ray data.

`effects.frag` traces reflections in screen space (`traceReflection`). A ray reflected from water often leaves the top of the screen, where there's nothing to sample, so `Scene::render_skyview` renders a second view above the normal one, into a separate framebuffer. Rays continue in that image (`traceSky`), and the view is skipped when no water is visible, which an occlusion query detects. Fresnel terms weight the result, and a reflected surface is given the same total haze as if it were viewed directly.

Stars, galaxies and nebulae

`OctTree`, `DensityTree` and `StarTree` (`Octree.cpp`) organize deep space. A noise expression defines density over the galaxy volume, and a color expression maps it to color. Stars are sampled from that density, with foreground and background groups, and drawn as GL points, with `galaxy.frag` shading the density. Selecting a star and going to it creates a `System`, which is how an effectively unlimited universe gets populated on demand.



A spiral galaxy generated from a density expression

The user interface

`VtxFrame` (`wx/VtxFrame.cpp`) is the main window, and a timer drives rendering on the canvas. The Scene Tree dialog (`VtxSceneDialog`) presents the object tree, and each object type has a tab class, such as `VtxPlanetTabs`, `VtxCloudsTabs` or `VtxRocksTabs`, which binds controls to object fields. Function dialogs edit noise, fractal, erosion and craters expressions, and image editors handle bands, images and textures. Controls generally update objects, then flag what changed through `set_changed_render` or `set_changed_detail`, and the next frame does the appropriate amount of work.

Known limitations

These are the main gaps in the current design:

- **Eclipses:** clouds and sky don't receive eclipse shadows, moons don't receive their planet's ring shadow, and only the first light casts analytic shadows.
- **Cloud shadows:** only volumetric clouds cast them. Sprites aren't shadowed, because they write type 0.
- **Adapt:** oscillation remains possible in some orbital views, and it's tied to the ID-buffer test.
- **Mesh seams:** adjacent marching-cubes cells at different depths can still produce gaps.
- **Platforms:** Mac support has not been tested.

Acknowledgments

The book that inspired this project is *Texturing & Modeling: A Procedural Approach*, by David S. Ebert, F. Kenton Musgrave, Darwyn Peachey, Ken Perlin and Steven Worley. Code examples from NVIDIA's *GPU Gems* series also informed the implementation.

Claude Opus, an AI model from Anthropic, helped implement many of the more recent features, including 3D rocks, asteroids and volumetric clouds.